NMEA-converter — Hardware Assembly Guide

Wiring a Waveshare ESP32-C6-LCD-1.47, an SN65HVD230 CAN transceiver, and a Wegmatt dAISy 2+ AIS receiver into a NMEA 0183 → NMEA 2000 bridge with on-screen target list.

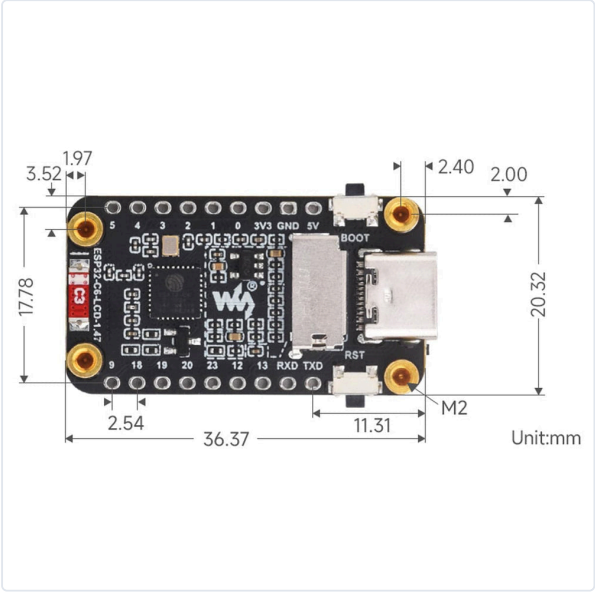
What you'll build

The converter reads `!AIVDM` sentences from a dAISy 2+ AIS receiver over a 3.3 V TTL UART link, decodes each AIS message, and re-emits it as the correct NMEA 2000 PGN onto the CAN bus through an SN65HVD230 transceiver. The onboard 172×320 ST7789 LCD shows decoded targets with type, speed, course and name; a sailboat icon flashes orange every time a PGN is sent.

Bill of materials

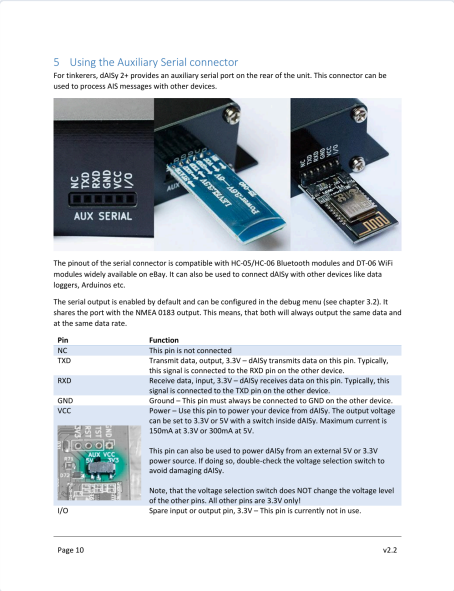
Item	Notes
Waveshare ESP32-C6-LCD-1.47 (non-touch SKU)	RISC-V single core, 160 MHz, 8 MB flash, 172×320 ST7789 LCD on SPI. USB-C.
SN65HVD230 CAN transceiver breakout	3.3 V-native CAN transceiver. The Waveshare variant has friendly RX/TX silkscreen labels and a 2-pole screw terminal for CAN_H / CAN_L.
Wegmatt dAISy 2+ AIS receiver	Dual-channel AIS receiver with USB and 3.3 V TTL AUX serial. Powered from its own USB.
Terminated NMEA 2000 backbone cable	Two 120 Ω terminators in place; ~60 Ω end-to-end across CAN_H / CAN_L when unpowered.
Hookup wire	Six conductors, 22–26 AWG. Solid core is fine for breadboard / dev-board headers.
USB-C cable for ESP32-C6	Powers and programs the converter from your computer.
USB-A cable for dAISy 2+	Powers the AIS receiver separately.

Waveshare ESP32-C6-LCD-1.47, front view. Photo via The Pi Hut product listing.



Rear of the same board with silkscreen pin labels. Note GPIO 19 on the bottom row — this is the pin we wire to the dAISy TX. Photo: The Pi Hut.

SN65HVD230 transceiver (Waveshare). Four-pin header on the left: `3.3V` / `GND` / `RX` / `TX`. Two-screw terminal on the right: `CANL` / `CANH`. Photo: The Pi Hut.



5 Using the Auxiliary Serial connector

For tinkerers, dAISy 2+ provides an auxiliary serial port on the rear of the unit. This connector can be used to process AIS messages with other devices.

The pinout of the serial connector is compatible with HC-05/HC-06 Bluetooth modules and DT-06 WiFi modules widely available on eBay. It can also be used to connect dAISy with other devices like data loggers, Arduinos etc.

The serial output is enabled by default and can be configured in the debug menu (see chapter 3.2). It shares the port with the NMEA 0183 output. This means, that both will always output the same data and at the same data rate.

Pin	Function
NC	This pin is not connected
TXD	Transmit data, output, 3.3V – dAISy transmits data on this pin. Typically, this signal is connected to the RXD pin on the other device.
RXD	Receive data, input, 3.3V – dAISy receives data on this pin. Typically, this signal is connected to the TXD pin on the other device.
GND	Ground – This pin must always be connected to GND on the other device.
VCC	Power – Use this pin to power your device from dAISy. The output voltage can be set to 3.3V or 5V with a switch inside dAISy. Maximum current is 150mA at 3.3V or 300mA at 5V. This pin can also be used to power dAISy from an external 5V or 3.3V power source. If doing so, double-check the voltage selection switch to avoid damaging dAISy. Note, that the voltage selection switch does NOT change the voltage level of the other pins. All other pins are 3.3V only!
I/O	Spare input or output pin, 3.3V – This pin is currently not in use.

Page 10 v2.2

dAISy 2+ rear, showing the AUX SERIAL header silkscreen. Pin order: `NC` `TXD` `RXD` `GND` `VCC` `I/O`. Source: Wegmatt dAISy 2+ AIS Receiver Manual v2.2, p.10.

Pin map at a glance

From	To	Purpose
ESP32-C6 **GND**	SN65HVD230 `GND` & dAISy 2+ AUX pin 4 (`GND`)	Common ground — wire **first**.
ESP32-C6 **3V3**	SN65HVD230 `3.3V`	Powers the transceiver. **Do not use 5V.**
ESP32-C6 **GPIO 4**	SN65HVD230 TX	CAN frames out of the ESP32, into the transceiver's driver input.

From	To	Purpose
ESP32-C6 **GPIO 5**	SN65HVD230 RX	Decoded CAN frames back into the ESP32.
ESP32-C6 **GPIO 19**	dAISy 2+ AUX pin 2 (TXD)	NMEA 0183 stream from the AIS receiver into the ESP32's UART1 RX.
SN65HVD230 CANH	Backbone CAN_H	Connect **last**, after bench tests pass.
SN65HVD230 CANL	Backbone CAN_L	Connect **last**.

> **Safe to know up front:** the dAISy 2+ manual states explicitly that the AUX TXD signal is 3.3 V regardless of the unit's VCC switch position. No level shifter or voltage divider is needed to feed it into the ESP32-C6.

Stage-by-stage assembly

Wire in this order. At each stage the worst-case failure mode is harmless. Connections to the live NMEA 2000 bus come last, after the source side is proven on the bench.

Stage 0 — Sanity check the dAISy TXD level (optional)

1. Power the dAISy 2+ via USB only. No other wires connected.
2. With a multimeter on DC volts, probe **TXD ↔ GND** on the AUX header.
3. Expect ~3.3 V (idle high). If you see 5 V or 0 V, stop and review the VCC selector switch position via the configuration menu before continuing.

Stage 1 — Unplug the ESP32 before any wiring

Always make and break connections while the ESP32 is **off USB**. Live wiring is how 3.3 V GPIOs get fried.

Stage 2 — Ground first, always

Common ground prevents floating-reference glitches and ESD events on the signal pins.

```
dAISy AUX pin 4 (GND) ─┐
                       ├── ESP32-C6 GND ── SN65HVD230 GND
                       └── (star or daisy-chain, either is fine)
```

Stage 3 — Power the transceiver from the ESP32

```
ESP32-C6 3V3 ──────► SN65HVD230 3.3V
```

The dAISy keeps its own USB power supply — **do not tie its VCC to anything**.

If your SN65HVD230 breakout has an `Rs` pin (slope-rate control), tie it to GND. Most boards have this already set internally; the Waveshare variant pictured does.

Stage 4 — Signal wires (ESP32 still unplugged)

```
dAISy AUX pin 2 (TXD)  ──────►  ESP32-C6 GPIO19
ESP32-C6 GPIO 4        ──────►  SN65HVD230 TX
SN65HVD230 RX          ──────►  ESP32-C6 GPIO 5
```

The `RX`/`TX` labels on the Waveshare transceiver are written from the microcontroller's point of view, so they line up one-to-one with the ESP32 pin names. Leave `CANH` / `CANL` unconnected

for now.

Stage 5 — Power up and verify the source side

1. Plug the dAISy USB cable (its LED should light).
2. Plug the ESP32 USB-C cable to your computer. The LCD wakes up showing the **AIS BRIDGE** title at ~40 % brightness, a gray `DAISY` pill, and a dim boat icon.
3. Within a few seconds of the dAISy emitting its first sentence, the `DAISY` pill should turn **solid green**. It latches — once green, stays green.
4. Each decoded AIS message triggers a ~0.4 s **orange flash on the boat icon**, and the `n2k:N` counter at the bottom-right increments by one.

If `DAISY` stays gray for a minute or more with the dAISy USB-powered and antenna connected, that's a source-side wiring or pin-assignment problem to debug before plugging into the bus.

Stage 6 — Connect to the NMEA 2000 backbone (only after Stage 5)

1. Power the backbone from a **12 V supply**. NMEA 2000 transceivers expect 12 V on the bus even if the ESP32 board itself stays on USB.
2. Confirm ~**60 Ω** across `CAN_H` / `CAN_L` on the unpowered backbone (both 120 Ω terminators in place, in parallel).
3. Wire:

   ```
   SN65HVD230 CANH ──► backbone CAN_H
   SN65HVD230 CANL ──► backbone CAN_L
   ```

4. Confirm the boat icon flashes and the `n2k` counter climbs as AIS targets come in. The `0183:M` count on the same line is total NMEA 0183 lines seen on UART — a higher number than `n2k` is normal, because multi-fragment AIS messages take two 0183 lines to make one decoded message.

Reading the screen

Each target row, left to right by importance:

Column	Meaning
Type	3-char vessel-type abbreviation, coloured by type (TNK=tanker red, CRG=cargo amber, PAX=passenger blue, FSH=fishing magenta, SAL=sailing green, TUG=tug olive, …).
Kts	Speed over ground in knots. `--` when unknown.
Deg	Course over ground in degrees. `---` when unknown.
Name	AIS-broadcast vessel name when known, otherwise last 8 digits of MMSI. The whole row is colour-coded by Class-A **nav status** (green = under way using engine, lime = under sail, orange = at anchor / moored, red = NUC / restricted / aground / SART, blue = engaged in fishing, black = Class B or unknown).

Targets that go silent for more than 5 minutes auto-evict from the list. The footer carries the live-target count, uptime, NMEA 2000 source address, and the running `n2k:N – 0183:M` message totals.

Power-saver behaviour

The CPU runs at 80 MHz and the backlight at ~40 % normally. After 5 minutes with no decoded AIS frames the backlight drops to ~5 %; any new AIS sentence or a press of the **BOOT** button on the C6 board restores it.

Troubleshooting

Symptom	Likely cause
LCD blank or backlight off	Firmware did not finish boot. Re-flash. Check USB-C cable is data-capable.
`DAISY` pill stays gray	No NMEA 0183 bytes reaching the ESP32. Check (a) dAISy is powered and antenna is connected; (b) the wire is on dAISy AUX **pin 2 (TXD)**, not RXD; (c) the ESP side lands on GPIO 19, not GPIO 20 or another GPIO; (d) common ground between dAISy and ESP.
Boat icon never flashes	NMEA 0183 sentences arriving but failing to decode. Open a serial monitor — the `0183:M` counter should be climbing. If it is but `n2k:N` stays at zero, the dAISy isn't seeing valid AIS messages (out of range or antenna issue).
n2k counter climbs but boat board sees nothing	Backbone wiring. Verify ~60 Ω across CAN_H/CAN_L with bus powered off, both terminators present, 12 V applied to the backbone.
ESP32 runs hot	Above ambient warm is normal for an ESP at 80 MHz with TFT backlight. If it's uncomfortable to touch, leave the auto-dim alone (it'll save ~30 % power after 5

Symptom	Likely cause
	min).

Disconnect order

1. Unplug ESP32 USB first.
2. Then any signal wire.
3. Then power (3.3 V).
4. Ground last.

This sequence avoids any case where a signal pin floats above a missing ground.
